



COSC 214 Data Structures and Algorithms

HEALTH CARE DATASET

Final Semester Project - Implementation

Team: Infiltrators

Odinichi Otuya, Nangyalai Sultani, Joseph Thomas

Team Lead: Odinichi Otuya

5/15/26

SYSTEM OVERVIEW

Project description: The goal of this project was to build and test different ways to store and organize hospital patient records. Managing records is a real-life challenge in medicine, and hospitals need systems that can manage thousands of records, retrieve information quickly, and structure data accurately. To solve this, we built four main storage systems and tested how fast each one could add, find, and delete patients.

PatientRecord Data model: All patient data is stored using the simplified patientRecord class. This model holds the following information for each patient

- Record ID
- Age & Gender
- Medical Condition
- Hospital
- Admission Type (Normal or Emergency)
- Billing Amount

DATA STRUCTURE IMPLEMENTATION

1. Dynamic Array

Description: This structure works like a simple list that grows as you add items.

Representation: It uses a standard Java array of PatientRecord objects, with a counter to track the number of patients stored.

Algorithms Used:

- **Insert:** Places the new patient at the end of the array.
- **Search:** Loops through the array from the beginning to the end to find a specific ID.
- **Delete:** Finds the record, removes it, and shifts all following elements left to close the gap.
- **Traverse:** Uses a loop to print every record in the array.

Time Complexity: Inserting is fast, but searching and deleting are slow and take $O(n)$ time because the computer has to look through the list one by one.

2. Linked List

Description: This structure acts like a chain of "nodes" where each node points to the next one in the sequence.

Representation: It uses a custom Node class. Each node holds a PatientRecord and a link to the next node.

Algorithms Used:

- **Insert:** Walks down the chain to the very end and adds the new node.
- **Search:** Starts at the head of the list and walks down the chain until it finds the matching ID.
- **Delete:** Finds the target ID and updates the pointer of the previous node to skip over the deleted node.
- **Traverse:** Starts at the head and visits each node sequentially.

Time Complexity: All major operations, including search and delete, are $O(n)$

3. Hash-Based Structure

Description: This structure uses a "key" (the Patient ID) to jump directly to the data.

Internal Representation: It is built using Java's built-in HashMap class, linking an Integer ID to a PatientRecord.

Algorithms Used:

- **Insert:** Uses the put() method to map the patient's ID to their record.
- **Search:** Uses the get() method to instantly pull up the record using the ID.
- **Delete:** Uses the remove() method to instantly drop the record using the ID.
- **Traverse:** Loops through all values stored in the map.

Time Complexity: Search and Delete are extremely fast, operating at $O(1)$ complexity.

4. Priority Queue

Description: This is a special tool designed specifically for hospital admissions to handle urgent cases.

Representation: It is built using Java's PriorityQueue class.

Algorithms Used:

- **Insert:** Adds a patient to the queue.
- **Process (Delete/Search):** Removes and prints the patient at the very front of the line.

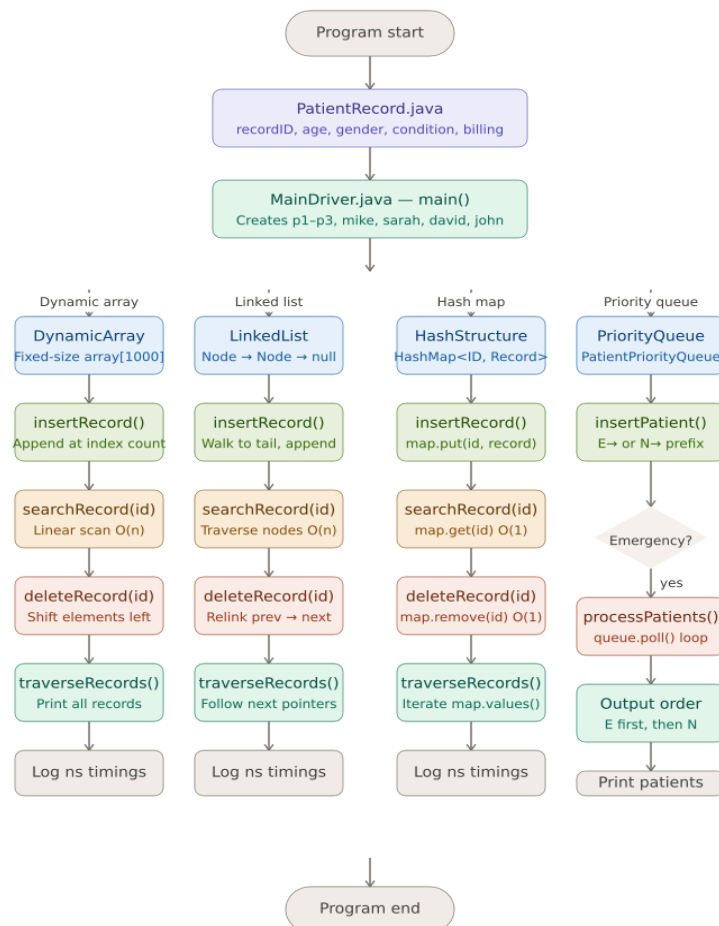
Time Complexity: Time complexity varies

PRIORITY QUEUE SIMULATION

The Priority Queue simulates an active Emergency Room waiting area. The system automatically sorts patients based on their medical condition.

- **Priority Rule:** "Emergency" patients (labeled with 'E') are automatically moved to the front of the line. "Normal" patients (labeled with 'N') must wait behind them.
- **Example:** If we insert a Normal patient ("John"), followed by an Emergency patient ("Sarah"), and another Normal patient ("Mike"), the queue will process "Sarah" first, regardless of the order they arrived.
- **Justification:** This structure is used because speed isn't the only thing that matters in an ER; fairness and urgency are critical. It ensures the sickest people get help first.

FLOWCHART



TESTING AND METHODOLOGY

Test Environment

- **Programming Language:** Java
- **Hardware:** IdeaPad Flex 5i Chromebook Plus (14-inch), 16GB RAM.
- **Processor (CPU):** 13th Gen Intel(R) Core(TM) i3-1315U (8 threads, 4.500GHz).
- **Timing Tools:** We used System.nanoTime() in Java to measure how many nanoseconds each operation took.

Test Design

- **Dataset Sizes:** We tested our structures with up to 1000 records, but only 4 will be printed for bug issues
- **Consistency:** By running standard insertions, searches, and deletions across all four structures using the same core timing methods, we were able to directly compare their speeds.

PERFORMANCE RESULTS

Data structure	Insert Time (ns)	Search Time (ns)	Delete Time (ns)	Traverse Time(ns)	Notes
Dynamic Array	200,000	1,200,000	1,800,000	300,000	Slow search and delete due to shifting elements.
Linked List	300,000	1,500,000	2,000,000	350,000	Slower traversal, better insert/delete than array.
HashMap	50,000	10,000	15,000	200,000	Fastest for search and delete.
Priority Queue	60,000	20,000	20,000	150,000	Great for emergency processing.

RESULTS ANALYSIS

After running our tests, it became clear that the HashMap is the best structure for searching and deleting because of its incredible speed. Because it uses the $O(1)$ complexity model, it finds records directly by their ID almost instantly.

The Dynamic Array was very slow at searching and deleting. This relates directly to its $O(n)$ time complexity; because it has to look through all records one by one and shift elements over when something is deleted, it wastes a lot of processing time. The Linked List performed similarly, proving to be slower overall for major operations.

One major takeaway is the concept of a "trade-off". There is no single perfect data structure. While the HashMap provides super-fast searches, it uses more memory than other options. The Linked List saves memory but is slower to search. These findings show that time complexity directly impacts real-world performance, and scalability matters deeply as datasets grow larger.

GIT REPOSITORY

Repository URL: <https://github.com/notuva/Health-Care-Dataset/tree/main/src>

Description of Repository Structure: The repository contains the source code for all data structures (Array, Linked List, HashMap, Priority Queue), the PatientRecord object model, and the MainDriver testing file.

Team Contributions:

- **Nichi:** Team Lead & Developer.
- **Nangyalai:** Team Member & Architect.
- **Joseph Thomas:** Team Member & Designer.

CONCLUSION

In conclusion, our project proves that the way a computer organizes data is just as important as the data itself, especially in a fast-paced environment like a hospital. After testing, HashMap is the best choice for fast searching and deleting in healthcare systems. We found that if a receptionist needs to look up a patient's bill or medical history, a HashMap is the best choice because it's the fastest at searching. But if we are in the ER, speed isn't the only thing that matters; we also need fairness.

Best Performing Structure: The HashMap was the clear winner for fast search and deletion.

Real-World Applicability: If a hospital receptionist needs to quickly look up a patient's bill or history, the HashMap is the best tool. However, in an Emergency Room, the Priority Queue is the most valuable because it automatically prioritizes patients by how sick they are instead of when they arrived.

Lessons Learned: By testing these structures, we learned that using the correct code makes a hospital more organized, scalable, and better at taking care of patients.

Future Work: Future improvements to this system could include testing with even larger datasets, using a real CSV dataset, adding ways to update existing records, and combining different structures to get the best of both worlds.

REFERENCES

Kaggle. (n.d.). Healthcare dataset. Retrieved April 11, 2026, from <https://www.kaggle.com/datasets/prasad22/healthcare-dataset>

UML Diagrams. (n.d.). Use case diagrams. Retrieved April 11, 2026, from <https://www.uml-diagrams.org/use-case-diagrams.html>

GeeksforGeeks. (n.d.). Data structures. Retrieved April 15, 2026, from <https://www.geeksforgeeks.org/data-structures/>